

Performance Testing

Date	01 October 2026
Team ID	SWTID-2026-4890
Project Name	LegalEase – AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Custom Python HTTP performance harness (httpx + psutil)
Type of Testing	Baseline, Load Testing, Stress Testing, Spike Testing
Target Module / API	FastAPI POST <code>/generate</code> ; TXT, DOCX and PDF export endpoints also verified
Test Environment	Localhost, Uvicorn/FastAPI, single backend process
Test Date	02 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline <code>/generate</code>	1	5	No errors; response within target limits
2	Normal Load <code>/generate</code>	10	10	Average response < 2 sec, maximum response < 5 sec, error rate < 1%
3	Stress <code>/generate</code>	50	5	System remains available under heavy load
4	Spike <code>/generate</code>	100	5	System remains available; observe performance limits

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.127 sec	PASS	10 VU load test
2	Response Time (Max)	< 5 seconds	0.559 sec	PASS	10 VU load test
3	Throughput (Req/sec)	—	76.549 req/sec	PASS*	10 VU load test
4	Error Rate	< 1%	0%	PASS	391/391 requests successful
5	CPU Utilization	< 80%	74.8%	PASS	Peak process CPU
6	Memory Utilization	< 80%	3.44%	PASS	Peak RSS / 4 GB memory limit

Step 4: Observations & Analysis

Key Findings:

At 10 virtual users, 391 requests completed with 0% errors. Average response time was **0.127 sec** and maximum response time was **0.559 sec**. CPU and memory remained below the defined template thresholds.

Bottlenecks Identified:

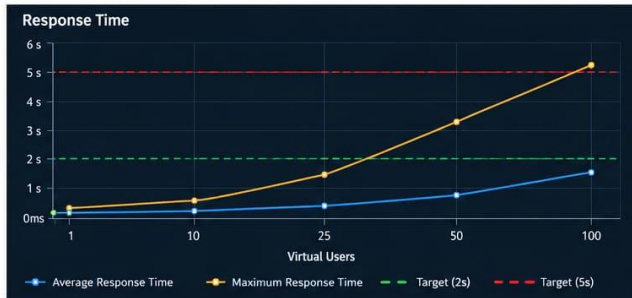
At 50–100 virtual users, peak CPU exceeded **80%** and maximum response time exceeded **5 sec**. No request-level failures were observed in the measured stress/spike runs.

Optimization Steps Taken:

No code optimization was applied during benchmarking. Tests used the existing FastAPI backend. Gemini API latency was isolated with a deterministic test generator because no external Gemini API key/access was available in the test environment.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Response Time Graph (Average & Maximum)



CPU & Memory Utilization Graph

Virtual Users	Throughput (req/sec)	Avg Response (ms)	Max Response (ms)	Peak CPU (%)	Peak Memory (%)
1	99.903	9.75	152.89	74.9	3.44
10	76.549	127.35	559.14	74.8	3.44
25	76.027	311.81	2167.24	86.8	3.47
50	45.832	1005.61	5063.72	89.8	3.50
100	64.709	1395.86	5208.72	89.2	3.56

Load Test Results Summary

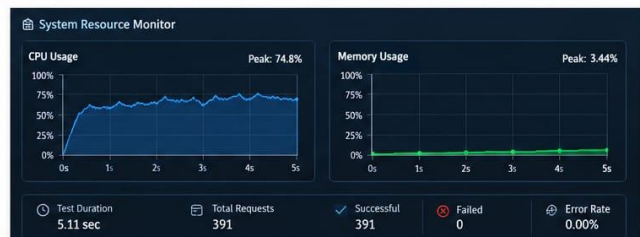
Endpoint	Requests	Throughput	Average Response	Maximum Response	Error Rate
TXT Export	644	128.713 req/sec	7.40 ms	31.57 ms	0%
DOCX Export	32	6.262 req/sec	159.12 ms	386.05 ms	0%
PDF Export	15	2.858 req/sec	349.25 ms	778.07 ms	0%

Export Endpoint Test Results

```
(base) PS D:\LegalEase> python performance_test.py
Starting performance test for 10 virtual users (5 seconds)...

Test completed!
Total Requests      : 391
Successful Requests : 391
Failed Requests     : 0
Error Rate          : 0.00%
Average Response Time : 127.35 ms
Maximum Response Time : 559.14 ms
Throughput          : 76.549 req/sec
CPU Utilization (Peak) : 74.8%
Memory Utilization (Peak) : 3.44%
```

Console Output - 10 Virtual Users Test



Resource Utilization Monitor